RESTful API Planning

1. API Selection

I chose the NHTSA Vehicle API because it provides information about different vehicles. Since I have used this API before, I wanted to work with it again and learn more about how websites get and share information. It follows REST principles by using URLs and HTTP methods to request data. Each request is handled separately, without depending on previous requests.

API: https://vpic.nhtsa.dot.gov/api/

2. Endpoint Mapping

These are the endpoints I plan to use:

Method	Endpoint	Purpose
GET	`/vehicles/GetVehicleVariableList?format=json`	Gets vehicle information from NHTSA.
POST	`/api/vehicle-searches`	Sends vehicle information to my planned Python backend.

GET will retrieve information, while POST will send information. Each endpoint identifies the resource being requested or submitted.

3. Data Flow

The user opens the webpage, and JavaScript Fetch sends a GET request to the NHTSA API. The API returns JSON data, which JavaScript processes and displays in a table.

The page will show a loading message while the information is being retrieved. If the request fails, an error message will appear.

Example Fetch request:

```
fetch("https://vpic.nhtsa.dot.gov/api/vehicles/GetVehicleVariableList?format=json", {
    headers: {
        "Accept": "application/json"
    }
})
.then(response => {
    if (!response.ok) {
        throw new Error("Unable to retrieve information.");
    }
    return response.json();
})
.then(data => console.log(data.Results))
.catch(error => console.log(error.message));
```

4. Dynamic Rendering and Styling

JavaScript will display the returned information in a table with the variable name, description, and type. I plan to use CSS to highlight information related to vehicle safety so it is easier to find.

5. Form Handling

The webpage will have a form with these fields:

- Vehicle Make – required.
- Vehicle Model – required.
- Model Year – required and must be a valid year.

JavaScript will check the information before sending it to the planned Python backend using POST.

Example JSON:

```
{
    "make": "Toyota",
    "model": "Camry",
    "model_year": 2024
}
```

The request will use the `Content-Type: application/json` header. The backend will receive and process the information. Building the backend is not required for this assignment.

6. Technology Stack

1. I plan to use HTML, CSS, and JavaScript to make the webpage. I will use Fetch API to get information from NHTSA. I also plan to use Python and Flask for the backend. JSON will help send and receive information, and GitHub will be used to save my work. I will use browser tools to check for problems and errors.
2. 7. Implementation Steps
3. Create a new GitHub repository.
4. Start building the webpage.
5. Connect the webpage to the NHTSA API.
6. Add messages to show when information is loading or when something goes wrong.
7. Show the information in a table.
8. Add colors to highlight certain information.
9. Create a form and check that the user enters the right information.
10. Plan how the webpage will connect to the Python backend.
11. Test the project and fix any problems.
12.

8. Testing and Deployment

I will check that the API returns information correctly and that the table, loading messages, and form validation work. I plan to store the project in GitHub. The webpage could be hosted online using GitHub Pages, while the Python backend would need separate hosting.

9. Conclusion

This project will help me better understand how websites communicate with other systems. It will also give me more practice working with REST APIs, JSON, and JavaScript.